C/C++ Memory Management Quiz - Complete Solutions

With Detailed Explanations

Q1. malloc() is declared in which header file?

Correct Answer: stdlib.h

Explanation: While some systems also provide malloc.h, stdlib.h is the standard C library header that declares malloc(), calloc(), realloc(), and free(). It is portable across all C/C++ implementations.

Q2. Which function initializes all allocated bytes to zero?

Correct Answer: calloc()

Explanation: calloc(number, size) allocates memory for an array and initializes all bytes to zero. malloc() does NOT initialize memory (contains garbage values). realloc() resizes existing memory without initialization.

Q3. Which function can increase or decrease the size of previously allocated memory?

Correct Answer: realloc()

Explanation: realloc(ptr, new_size) attempts to resize the memory block pointed to by ptr. If successful, it may move the data to a new location and returns the new address. It preserves existing content up to the lesser of old and new sizes.

Q4. Which operator is used for dynamic memory allocation in C++?

Correct Answer: new

Explanation: C++ introduces 'new' and 'delete' operators for dynamic memory allocation/deallocation. They are type-safe, call constructors/destructors, and are preferred over malloc()/free() in C++.

Q5. Memory allocated using new resides in

Correct Answer: Heap

Explanation: The 'new' operator allocates memory from the free store (heap). Stack memory is used for local variables and function call frames, while registers and cache are hardware-managed storage.

Q6. Local variables are usually stored in

Correct Answer: Stack

Explanation: Local/automatic variables are allocated on the stack. The stack grows and shrinks with function calls and returns. Heap is for dynamic allocation, ROM is read-only, and global memory is for static/global variables.

Q7. Which statement correctly deallocates an array allocated using new?

Correct Answer: delete[] arr;

Explanation: When allocating arrays with 'new[]', you MUST use 'delete[]' to deallocate. Using plain 'delete' causes undefined behavior (only the first element's destructor is called). free() should never be used with new-allocated memory.

Q8. Which statement is true regarding malloc()?

Correct Answer: Returns void*

Explanation: malloc() returns a void* pointer to the allocated memory. It does NOT initialize memory, call constructors, or call destructors. In C++, the void* must be cast to the appropriate type.

C/C++ Memory Management Quiz - Complete Solutions

With Detailed Explanations

Q9. Which statement is true regarding new?

Correct Answer: Calls constructor

Explanation: In C++, 'new' automatically calls the object's constructor after allocation. It returns a typed pointer (not void*), can allocate arrays, and is C++-specific (does not exist in C).

Q10. Memory leak occurs when

Correct Answer: Allocated memory is never released

Explanation: A memory leak happens when dynamically allocated memory is no longer reachable (no pointers reference it) but was never freed. Freeing twice is a double-free error, and NULL pointers don't cause leaks by themselves.

Q11. Which pointer refers to already deallocated memory?

Correct Answer: Dangling pointer

Explanation: A dangling pointer points to memory that has been freed/deallocated. Using it causes undefined behavior. A null pointer points to nothing (address 0), a wild pointer is uninitialized, and void* is a generic pointer type.

Q12. Which pointer is never initialized?

Correct Answer: Wild pointer

Explanation: A wild pointer is declared but never initialized, containing a random garbage address. Null pointers are explicitly set to NULL/nullptr, dangling pointers once pointed to valid memory, and smart pointers are managed objects.

Q13. Which is true about nullptr in C++?

Correct Answer: It is a type-safe null pointer

Explanation: nullptr (C++11+) is a type-safe null pointer constant of type std::nullptr_t. Unlike NULL (which is often just 0), nullptr cannot be confused with integers, preventing dangerous overload resolution errors.

Q14. Which memory allocation is generally faster?

Correct Answer: Stack allocation

Explanation: Stack allocation is extremely fast - just decrementing the stack pointer. Heap allocation requires complex bookkeeping, searching for suitable blocks, and potential system calls, making it significantly slower.

Q15. Which memory is automatically reclaimed after a function returns?

Correct Answer: Stack

Explanation: Stack memory (local variables, parameters) is automatically freed when a function returns via the stack pointer reset. Heap memory must be explicitly freed by the programmer or garbage collector.

Q16. What happens if delete is applied twice to the same pointer?

Correct Answer: Undefined behavior

Explanation: Double-delete is undefined behavior. The heap manager may crash, corrupt memory, or appear to work. After delete, the pointer becomes dangling. Best practice: set pointer to nullptr after deletion (delete on nullptr is safe and does nothing).

Q17. Which memory region commonly suffers from fragmentation?

Correct Answer: Heap

Explanation: Heap fragmentation occurs when allocated and freed blocks create small non-contiguous holes. Stack memory is contiguous and LIFO, so it doesn't fragment. Registers and ROM are not subject to fragmentation.

Q18. Which fragmentation occurs because free memory is scattered in small blocks?

Correct Answer: External fragmentation

Explanation: External fragmentation: Total free memory is sufficient, but it's scattered in small non-contiguous blocks, so large allocation requests fail. This happens when variable-sized blocks are allocated and freed on the heap.

Q19. Which fragmentation occurs due to unused space inside an allocated block?

Correct Answer: Internal fragmentation

Explanation: Internal fragmentation occurs when more memory is allocated than requested (due to alignment requirements or fixed block sizes), leaving unused space INSIDE the allocated block. The allocator cannot use this space for other requests.

Q20. Which of the following uses dynamic memory internally?

Correct Answer: All of these

Explanation: std::vector, std::string, and std::list all allocate dynamic heap memory internally to grow and shrink as needed. std::vector uses contiguous dynamic arrays, std::list uses node-based allocation, and std::string typically uses dynamic buffers.

Q21. Stack follows which allocation strategy?

Correct Answer: LIFO

Explanation: Stack follows Last-In-First-Out (LIFO). The last item pushed (most recent function call/local variable) is the first one popped when the function returns. This makes stack management extremely efficient.

Q22. Heap memory generally supports

Correct Answer: Dynamic allocation

Explanation: Heap supports dynamic (runtime) allocation with variable sizes and lifetimes. Stack is LIFO and fixed at compile time for each frame, while global/static variables have fixed allocation.

Q23. Which situation may cause stack overflow?

Correct Answer: Infinite recursion

Explanation: Infinite recursion causes unlimited stack growth, eventually exhausting the stack space. Correct delete usage, cout, and simple int declarations do not cause stack overflow.

Q24. Recursive functions primarily consume

Correct Answer: Stack memory

Explanation: Each recursive call pushes a new stack frame containing parameters, local variables, and return address. Deep recursion consumes significant stack space. Heap is used only if the recursive function explicitly allocates dynamic memory.

Q25. What is printed conceptually by int *p = new int; cout << p;?

Correct Answer: Address of allocated heap memory

Explanation: Printing a pointer variable (without dereferencing) outputs the memory address it stores. Since 'new int' returns the address of heap-allocated memory, this prints that heap address, not the value (which is uninitialized/garbage).

Q26. What is printed by int x=10; int *p=&x; cout<<*p;?

Correct Answer: 10

Explanation: The * operator dereferences the pointer, accessing the value at the address stored in p. Since p points to x (which is 10), *p evaluates to 10. Without *, it would print the address of x.

Q27. Which operator returns the address of a variable?

Correct Answer: &

Explanation: The unary & operator (address-of) returns the memory address of its operand. This is distinct from the binary & (bitwise AND). The * operator dereferences, % is modulo, and # is preprocessor directive.

Q28. Which operator accesses value stored at an address?

Correct Answer: *

Explanation: The unary * operator (dereference) accesses the value stored at the address contained in a pointer. It is the inverse of &. & returns address from value, * returns value from address.

Q29. Which is true about delete p;?

Correct Answer: It deletes heap memory pointed by p

Explanation: delete p deallocates the heap memory that p points to and calls the destructor. It does NOT delete the pointer variable itself (p remains on stack or wherever it was declared). It also does NOT automatically set p to NULL.

Q30. Can free() be safely used on memory allocated using new?

Correct Answer: No, it is undefined behavior

Explanation: Never mix malloc/free with new/delete. They may use different heap managers or metadata. free() does not call destructors, and using free() on new-allocated memory causes undefined behavior, potentially corrupting the heap.

Q31. Can delete be safely used on memory allocated using malloc()?

Correct Answer: No, it is undefined behavior

Explanation: Using delete on malloc()-allocated memory is undefined behavior. delete expects metadata/format from new, which may differ from malloc's implementation. Always pair malloc with free, and new with delete.

Q32. Which memory allocation happens at runtime?

Correct Answer: Dynamic allocation

Explanation: Dynamic allocation (malloc/new) occurs at runtime when the program requests memory. Static and compile-time allocation are determined before/during compilation, and preprocessor allocation is not a real memory concept.

Q33. Which memory allocation size is usually fixed before program execution?

Correct Answer: Static allocation

Explanation: Static allocation (global variables, static locals, string literals) has size determined at compile time and exists for the entire program lifetime. Dynamic allocation sizes are determined at runtime.

Q34. Which of the following typically requires a pointer to access allocated memory?

Correct Answer: All of these

Explanation: malloc() and calloc() return void* (or typed pointer in C++). new returns a typed pointer. All dynamic allocation mechanisms return a pointer that must be stored and used to access the allocated memory.

Q35. Which language introduced new/delete operators for object allocation?

Correct Answer: C++

Explanation: C++ introduced new and delete operators. C uses malloc()/calloc()/realloc()/free(). Assembly requires manual memory management, and SQL is a query language unrelated to system memory allocation.

Q36. Static binding is also known as

Correct Answer: Early binding

Explanation: Static/early binding resolves function calls at compile time based on the variable's declared type. The compiler knows exactly which function to call, enabling optimizations but no runtime flexibility.

Q37. Dynamic binding is also known as

Correct Answer: Late binding

Explanation: Dynamic/late binding resolves function calls at runtime based on the actual object's type. This enables polymorphism but has slight overhead due to virtual table lookups.

Q38. Virtual functions in C++ use

Correct Answer: Late binding

Explanation: Virtual functions enable late binding through vtables. The actual function called is determined at runtime based on the dynamic type of the object, not the static type of the pointer/reference.

Q39. Function overloading is an example of

Correct Answer: Compile-time polymorphism

Explanation: Overloading (same name, different parameters) is resolved at compile time by the compiler. It is static/compile-time polymorphism. The compiler selects the appropriate function based on argument types.

Q40. Function overriding with virtual functions is an example of

Correct Answer: Runtime polymorphism

Explanation: Overriding a virtual function in a derived class enables runtime polymorphism. The call is resolved at runtime via the vtable, allowing derived class implementations to be called through base class pointers.

Q41. Which architecture stores instructions and data in the same memory?

Correct Answer: Von Neumann architecture

Explanation: Von Neumann (Princeton) architecture uses a single shared memory and bus for both instructions and data. This is the dominant architecture in modern general-purpose computers (x86, ARM).

Q42. Which architecture separates instruction memory and data memory?

Correct Answer: Harvard architecture

Explanation: Harvard architecture uses separate memories and buses for instructions and data, allowing simultaneous access. Modern modified Harvard architectures (ARM Cortex-M, DSPs) cache unified memory but maintain separate L1 caches.

Q43. Heap memory is managed mainly by

Correct Answer: Runtime system and operating system support

Explanation: Heap management is a collaboration between the language runtime (C/C++ standard library allocator) and the OS. The runtime requests large chunks from the OS via brk/mmap and subdivides them for program requests.

Q44. Which STL container performs dynamic memory allocation automatically?

Correct Answer: vector

Explanation: std::vector automatically manages dynamic memory, reallocating when capacity is exceeded. While string and list also use dynamic memory, vector is the quintessential example of automatic dynamic resizing in STL.

Q45. Smart pointers were introduced to help solve

Correct Answer: Memory leak and ownership issues

Explanation: Smart pointers (unique_ptr, shared_ptr, weak_ptr) automate memory management, preventing leaks by ensuring delete is called exactly once when the last owner goes out of scope. They also clarify ownership semantics.

Q46. Which smart pointer allows only one owner?

Correct Answer: unique_ptr

Explanation: unique_ptr enforces exclusive ownership - only one pointer can own the object. It cannot be copied, only moved. When the unique_ptr goes out of scope, it automatically deletes the owned object.

Q47. Which smart pointer allows multiple owners?

Correct Answer: shared_ptr

Explanation: shared_ptr uses reference counting to allow multiple pointers to own the same object. The object is deleted only when the last shared_ptr is destroyed. This enables complex shared ownership patterns.

Q48. Which smart pointer cannot be copied?

Correct Answer: unique_ptr

Explanation: unique_ptr cannot be copied (copy constructor is deleted) because it enforces single ownership. It can only be moved via std::move(). shared_ptr can be copied, and weak_ptr can also be copied.

Q49. Which smart pointer helps observe shared_ptr without owning it?

Correct Answer: weak_ptr

Explanation: weak_ptr holds a non-owning reference to an object managed by shared_ptr. It can check if the object still exists (lock()) without affecting its lifetime, preventing circular reference problems.

Q50. Which one is NOT a disadvantage of Dynamic Memory Allocation?

Correct Answer: Flexible memory usage

Explanation: Flexible memory usage is actually the primary ADVANTAGE of DMA - allocating exactly what you need, when you need it. The disadvantages include memory leak risk, fragmentation, and slower performance compared to stack allocation.